Schema Manipulation Outside Migrations

All the migration methods described so far in this chapter are also available as methods on Active Record connection objects and so are accessible within the models, views, and controllers of a Rails application.

For example, you might have discovered that a particular long-running report runs a lot faster if the `orders` table has an index on the `city` column. But that index isn't needed during the day-to-day running of the application, and tests have shown that maintaining it slows the application appreciably.

Let's write a method that creates the index, runs a block of code, and then drops the index. This could be a private method in the model or could be implemented in a library:

```
def run_with_index(*columns)
  connection.add_index(:orders, *columns)
  begin
    yield
  ensure
    connection.remove_index(:orders, *columns)
  end
end
```

The statistics-gathering method in the model can use this as follows:

```
def get_city_statistics
  run_with_index(:city) do
    # .. calculate stats
  end
end
```

What We Just Did

While we had been informally using migrations throughout the development of the Depot application and even into deployment, in this chapter we saw how migrations are the basis for a principled and disciplined approach to configuration management of the schema for your database.